

2^η Σειρά Ασκήσεων Προηγμένες Εφαρμογές Ψηφιακής Σχεδίασης

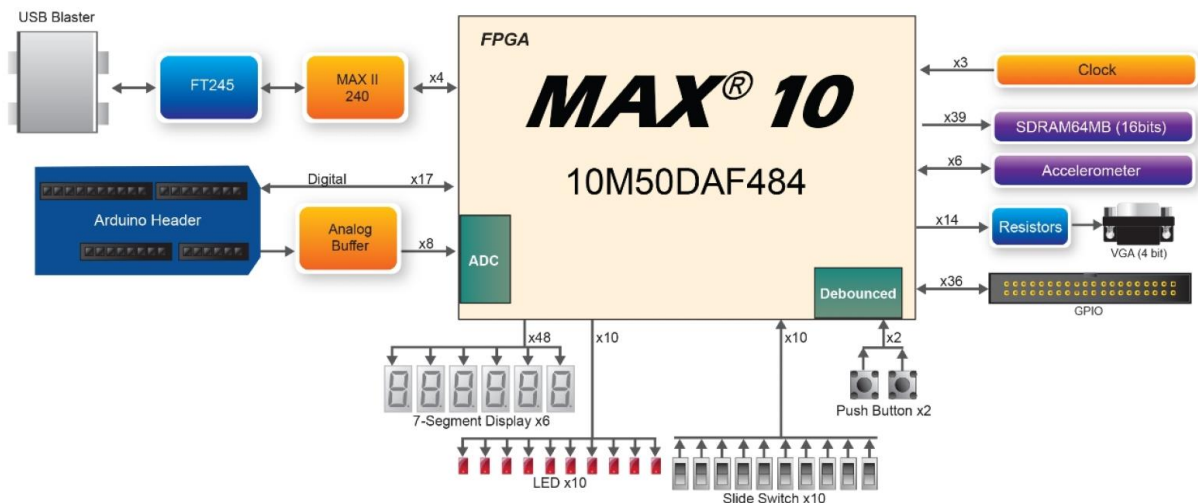
Ημερομηνία Παράδοσης: 19/12/2022 23.00μ.μ.

Σύστημα Παράδοσης: courses.cs.ihu.gr

Μορφή αρχείου: AEM.pdf

Σκοπός Σειράς Ασκήσεων:

Ο σκοπός αυτής της σειράς ασκήσεων είναι οι φοιτητές να περιγράψουν σχεδιάσεις, που γνωρίζουν από το μάθημα Ψηφιακής Σχεδίασης, σε **SystemVerilog** και με στοχευμένη τεχνολογία **FPGA/CPLD MAX 10 (10M50DAF484C6GES)**.



Οι φοιτητές θα υλοποιήσουν διάφορα ακολουθιακά κυκλώματα, όπως: Flip Flop, καταχωρητές, μετρητές και άλλα, με διάφορες τεχνικές περιγραφής υλικού (HDL) και με χρήση διάφορων προγραμμάτων δοκιμών (**Testbench**).

Επιπρόσθετα, οι φοιτητές θα προσομοιώσουν σχεδιάσεις SystemVerilog σε **επίπεδο RTL (Register Transfer Level)**.

Οι ασκήσεις θα υλοποιηθούν με το λογισμικό **Quartus Prime** και τον προσομοιωτή **ModelSim** ώστε οι φοιτητές να εξοικειωθούν με τις σύγχρονες ροές ψηφιακής σχεδίασης (Digital Design Flows).

Προσοχή: Η SystemVerilog είναι case-sensitive!!!

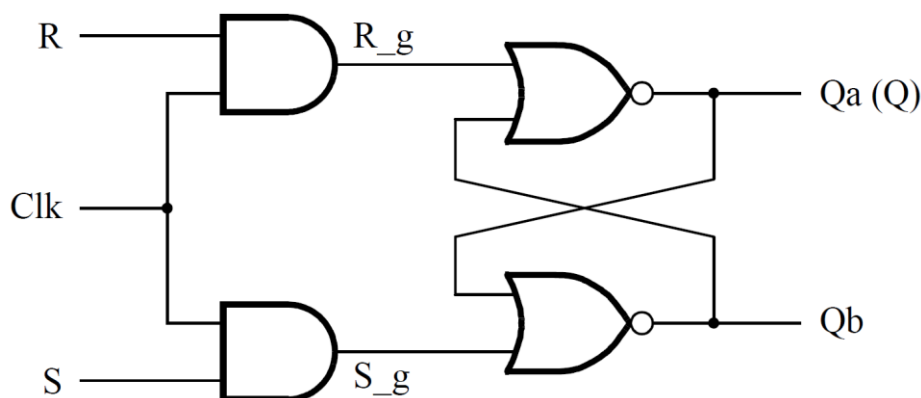
✓ Μέρος I ✓

Το FPGA ολοκληρωμένο **MAX 10 (10M50DAF484C6GES)** του **Board MAX 10 DE10 - Lite**, όπως και όλα τα FPGAs, περιέχουν στοιχεία μνήμης (Flip Flops) που είναι στη διάθεση του σχεδιαστή. Αν θέλουμε να υλοποιήσουμε το στοιχείο μνήμης χωρίς να χρησιμοποιήσουμε τα διαθέσιμα Flip Flops θα πρέπει να περιγράψουμε με συγκεκριμένο τρόπο τη σχεδίαση μας.

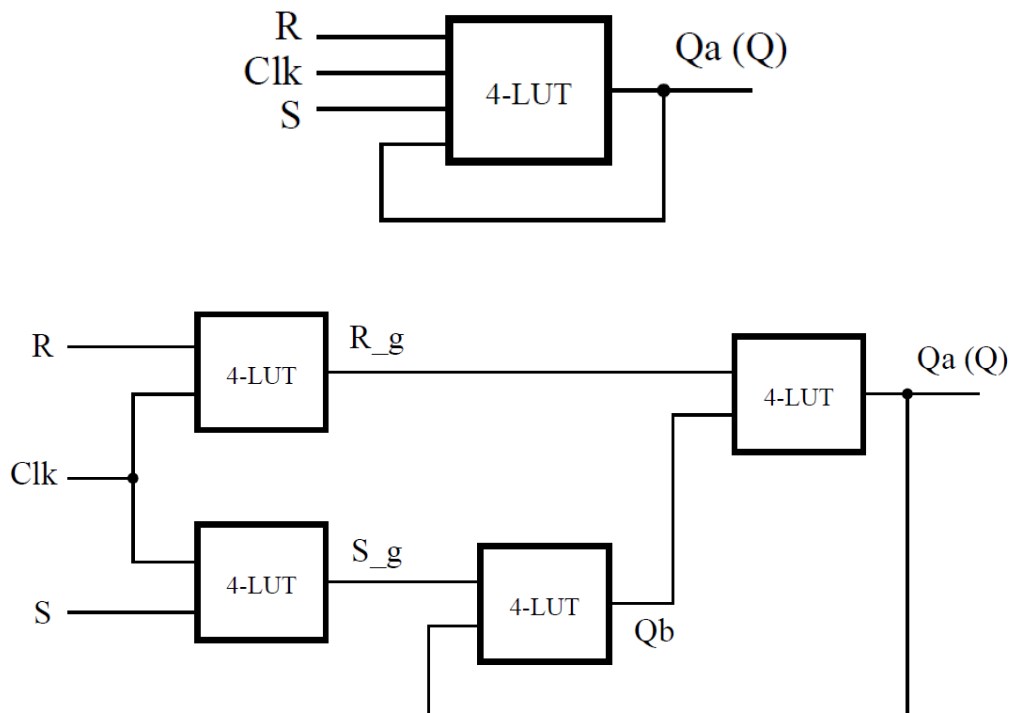
Στην περίπτωση που έχουμε ένα σύγχρονο μανδαλωτή RS (gated RS), όπως παρουσιάζεται στον παρακάτω σχήμα, και θέλουμε να το υλοποιήσουμε χωρίς την χρήση μόνο μιας τεσσάρων (4) εισόδων LUT θα πρέπει να κάνουμε χρήση της SystemVerilog οδηγίας ***/* synthesis keep */*** ώστε να περιγράψουμε με αναθέσεις το κύκλωμα και τις αναδράσεις που θα υλοποιηθεί χωρίς βελτιστοποίηση κατά το στάδιο της Σύνθεσης Κυκλωμάτων του λογισμικού Quartus Prime. Με τη διαδικασία αυτή το κύκλωμα θα υλοποιηθεί με τέσσερις 4-LUTs.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (rslatch.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας με τη βοήθεια των εντολών που είναι διαθέσιμες. Να αποθηκεύσετε τις εντολές σε ένα **αρχείο .do (Save Transcript As... rslatch.do)** και να εκτελέσετε ξανά την προσομοίωση (**do rslatch.do**).
6. Επαναλάβετε όλα τα βήματα χωρίς το **/* synthesis keep */**.
7. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.



Εικόνα 1. Ο μανδαλωτής RS.



Εικόνα 2. Η υλοποίηση του μανδαλωτή με ή χωρίς synthesis keep.

```
-- A gated RS latch described the hard way
module rslatch(input  logic R,S,Clk,
               output logic Q);

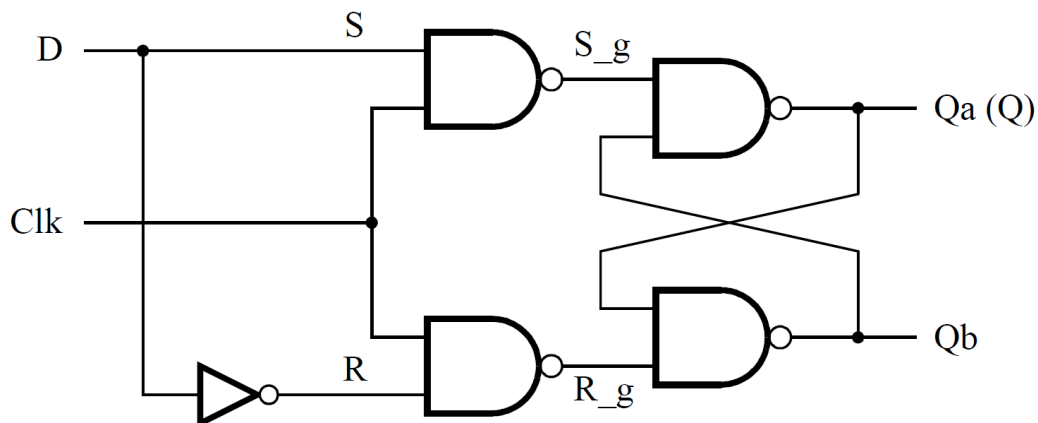
    logic R_g,S_g,Qa,Qb /* synthesis keep */;

    assign R_g = R && Clk;
    assign S_g = S && Clk;
    assign Qa = ~(R_g || Qb);
    assign Qb = ~ (S_g || Qa);
    assign Q = Qa;
endmodule

-- inputs for simulation
force Clk 1 0, 1 40, 0 60, 1 80, 0 100
force S 1 0, 0 60, 1 100
force R 0 0, 1 60, 0 100
run 100
```

Μέρος II

Να υλοποιήσετε ένα σύγχρονο μανδαλωτή D (gated D Latch) όπως παρουσιάζεται στο σχήμα.



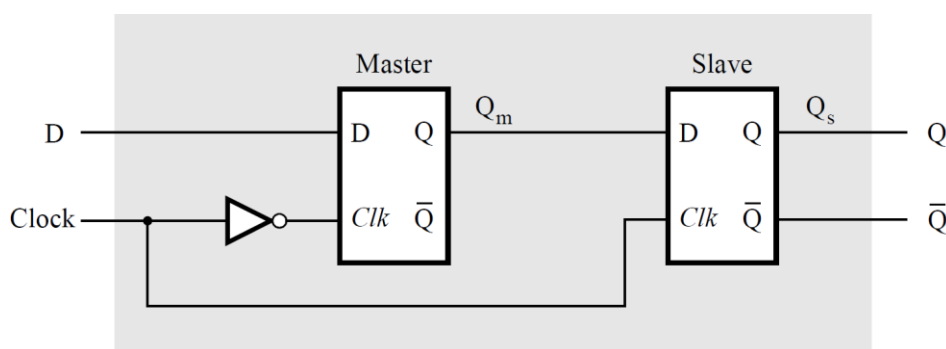
Εικόνα 3. Ο σύγχρονος μανδαλωτής D.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (gdlatch.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας με τη βοήθεια των εντολών που είναι διαθέσιμες. Να αποθηκεύσετε τις εντολές σε ένα **αρχείο .do (Save Transcript As... gdlatch.do)** και να εκτελέσετε ξανά την προσομοίωση (**do gdlatch.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος III

Να υλοποιήσετε έναν Αφέντη-Σκλάβο μανταλωτή D (master-slave D flip flop). Για να υλοποιήσουμε το flip flop θα καλέσετε δύο στιγμιότυπα του latch του Μέρους II και θα τα συνδέσετε όπως παρουσιάζεται παρακάτω.



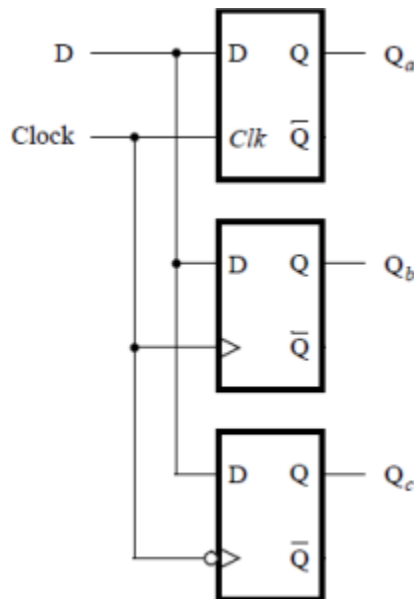
Εικόνα 4. Ο Αφέντης-Σκλάβος μανταλωτή D.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (msdff.sv)** για τον κώδικα που θα καλεί τα **gdlatch** και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Καλέστε τον προσομοιωτή **Altera-ModelSim** για προσομοίωση **RTL** επιπέδου.
5. Με εντολές command line του **Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας με τη βοήθεια των εντολών που είναι διαθέσιμες. Να αποθηκεύσετε τις εντολές σε ένα **αρχείο .do (Save Transcript As... msdff.do)** και να εκτελέσετε ξανά την προσομοίωση (**do msdff.do**).
6. **Παραδοτέα:** κώδικας SV, εικόνα από τον Netlist Viewer, αρχείο .do, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος IV

Να υλοποιήσετε τα στοιχεία μνήμης (σύγχρονος μανταλωτής D, ακμοपुरοδότητο D Flip Flop στην θετική ακμή, ακμοपुरοδότητο D Flip Flop στην αρνητική ακμή) που παρουσιάζονται στο παρακάτω. Για τις περιγραφές των στοιχείων μνήμης θα χρησιμοποιήσετε ως βάση την εντολή **always** όπως στο παράδειγμα που δίνετε αλλά με τις παραλλαγές της (**always_ff**, **always_latch**).



Εικόνα 5. Τα τρία στοιχεία μνήμης σε κοινό κύκλωμα.

```
-- A gated D latch described with always
module gdlatchalw(input logic D, Clock,
                  output logic Q);

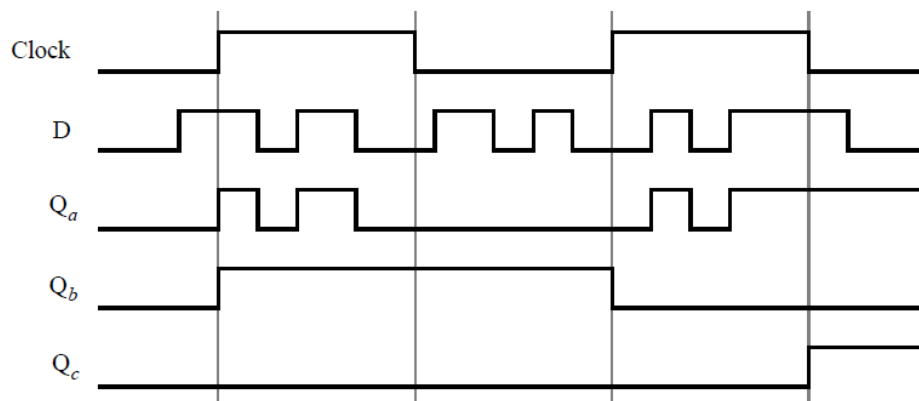
    always@ (Clock,D)
        if (Clock) Q<=D;
endmodule
```

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τις σχεδιάσεις **SystemVerilog** (**dalwlatc**, **pedff**, **nedff**) και ένα αρχείο **SystemVerilog** (**lab2p4**) για τη συνολική σχεδίαση.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας (με ένα απλό αρχείο δοκιμών – μόνο εισόδους).
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Πιο αναλυτικά τα επιμέρους βήματα προσομοίωσης (ΒΗΜΑ 4) με **SystemVerilog Testbench** στο **Altera-ModelSim** μέσα από το **Quartus Prime**:

```
-- Περιγραφή SystemVerilog για τη σχεδίαση DUT (lab2p4.sv)
-- Περιγραφή SystemVerilog Testbench για τη DUT (lab2p4_tb.sv)
-- Ορισμός της σχεδίασης ως top-level entity (lab2p4_tb)
-- Analysis & Elaboration της σχεδίασης lab2p4_tb
-- Έλεγχος των περιγραφών για σφάλματα / αποσφαλμάτωση
-- Σε περίπτωση που δεν έχουμε σφάλματα:
-- Ορισμός της σχεδίασης DUT (lab2p4) ως top-level entity
-- Analysis & Synthesis της σχεδίασης lab2p4
-- Quartus Prime: Assignments > Settings... > Simulation
-- NativeLink Settings > Compile Test bench: > add file
-- Test benches... > New > Test Bench Name: lab2p4_tb
-- Test Bench and Simulation Files > ... > Add > lab2p4_tb.sv
-- (HDL_version: SystemVerilog_2005)
-- Πατάμε ok > ok > Apply > ok
-- Tools > Run Simulation Tool > RTL Simulation
-- Έλεγχος των αποτελεσμάτων στο παράθυρο κυματομορφών Altera-ModelSim.
```



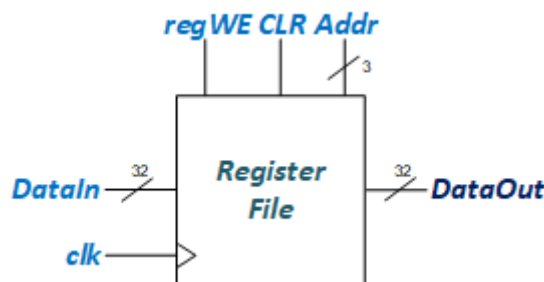
Εικόνα 6. Ενδεικτικό παράθυρο προσομοίωσης.

Μέρος V

Αρχικά, να υλοποιηθεί ένας καταχωρητής D Flip Flop με παραμετρικό N και με σύγχρονη είσοδο CLR (καθαρισμός καταχωρητή). Η περιγραφή να γίνει με χρήση εντολής always ff και παραμετρικής δήλωσης #(parameter). Για τη συγκεκριμένη άσκηση να υλοποιηθεί ένας καταχωρητής 32 bit (η τιμή N=32).

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (regN.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας.
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.



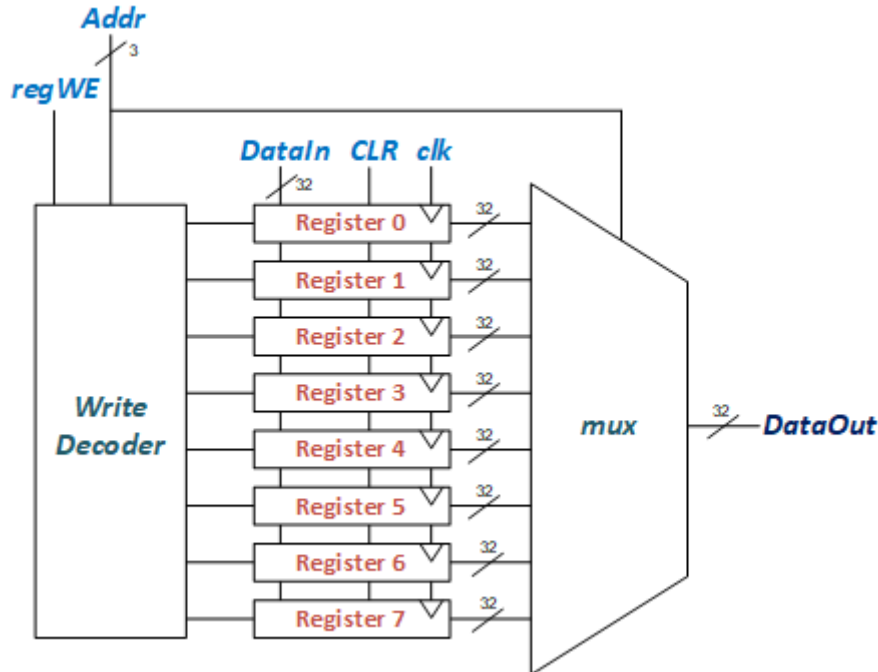
Εικόνα 7. Το μπλοκ διάγραμμα του Αρχείου Καταχωρητών.

Στη συνέχεια, να υλοποιήσετε ένα κύκλωμα **Αρχείου Καταχωρητών με σύγχρονη θύρα μονής εγγραφής και ασύγχρονη θύρα μονής ανάγνωσης (synchronous write single port and asynchronous read single port Register File)**, όπως απεικονίζεται στην εικόνα 7, που θα έχει οκτώ (8) καταχωρητές των 32bit. Αρχικά, η σχεδίαση θα υλοποιηθεί με **δομικό τρόπο (structural)** και σύμφωνα με το διάγραμμα της εικόνας 8. Ιδιαίτερη προσοχή πρέπει να δώσετε στη συνδυαστική λογική που πρέπει να έχει ο αποκωδικοποιητής για να ενεργοποιείται η εγγραφή με το **regWE**.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση (top-level) σε ένα αρχείο **SystemVerilog (SregFile.sv)** και για τα υποκυκλώματα δημιουργήστε: **WRDecode.sv**, **regN.sv (N=32)**, **mux3281.sv** και συμπεριλάβετε τα στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).

4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας.
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.



Εικόνα 8. Το δομικό διάγραμμα του Αρχείου Καταχωρητών.

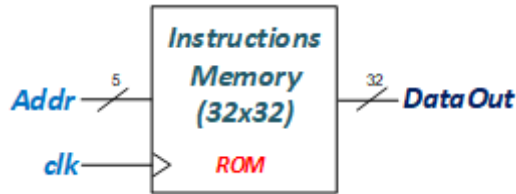
Τέλος, να υλοποιηθεί με **Behavioral SystemVerilog** το Αρχείο Καταχωρητών.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (regFile.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας.
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Μέρος VI

Αρχικά, να υλοποιηθεί μια **Μνήμη Εντολών** που θα διαθέτει **32** θέσεις μεγέθους λέξης (**word**). Η υλοποίηση της θα γίνει με **SystemVerilog Behavioral**. Η μνήμη εντολών θα είναι μια μνήμη **ROM (Read Only Memory)** μιας θύρας ανάγνωσης και με σύγχρονό ρολόι.

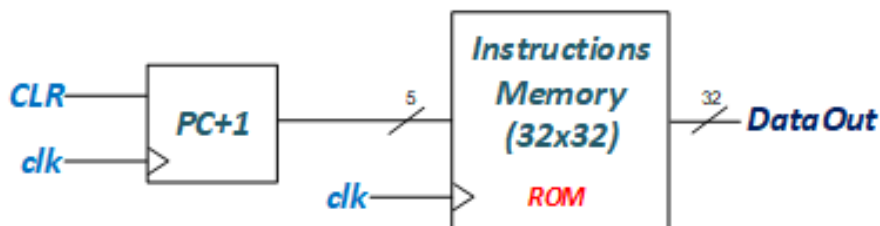


Εικόνα 9. Το δομικό διάγραμμα της Μνήμης Εντολών.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (ROM.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Η μνήμη να αρχικοποιείται με αρχείο .txt. (Εικόνα 13)
5. Παραδοτέα: κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Στη συνέχεια για να έχουμε προσπέλαση στην μνήμη εντολών να υλοποιηθεί ένας **μετρητής προγράμματος (Program Counter)** με **σύγχρονη είσοδο CLR (μηδενισμός PC)**. Η περιγραφή να γίνει με χρήση εντολής **always ff** και παραμετρικής δήλωσης **#(parameter)**. Ο μετρητής έχει εισόδους **CLR** (μηδενίζει τον μετρητή στο '1'), **clk** και έξοδο **Q** των πέντε (5) bit. Να συνδυαστούν τα δύο κυκλώματα με δομικό τρόπο όπως παρουσιάζεται στην εικόνα 10.



Εικόνα 10. Το δομικό διάγραμμα της προσπέλασης μνήμης εντολών με τον μετρητή προγράμματος.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (PCROM.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.

3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Η μνήμη να αρχικοποιείται με αρχείο **command mem.txt**.
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

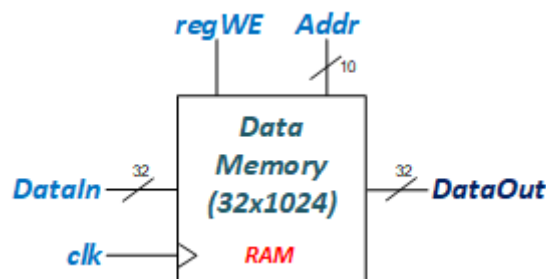
Το κύκλωμα **PCROM** να υλοποιηθεί με **Quartus IP Catalog (Altera Megafunctions)** όπως έχουμε διδαχθεί στο μάθημα και να επαναληφθούν όλα τα βήματα προσομοίωσης (**IPPCROM.sv**). Η μνήμη να αρχικοποιείται με αρχείο **command_mem.mif (MIF – Memory Initialization File)**. Τι παρατηρείται στην υλοποίηση της μνήμης ROM σε σχέση με την πρώτη υλοποίηση;

Οι εντολές προγράμματος που θα φορτώνουν και τα δύο προγράμματα στη μνήμη θα είναι από τον assembly κώδικα που είναι στο ΠΑΡΑΡΤΗΜΑ.

Τέλος, να υλοποιηθεί μια **μνήμη δεδομένων (Data Memory)** που θα έχει μία θύρα ανάγνωσης/εγγραφής. Αν η έγκριση εγγραφής (WE) της θύρας είναι ενεργοποιημένη (έχει την τιμή 1), τότε η θύρα εγγράφει τα δεδομένα WD στη διεύθυνση A κατά την ανερχόμενη ακμή του ρολογιού. Αν η έγκριση εγγραφής έχει τιμή 0, τότε διαβάζει δεδομένα από τη διεύθυνση A και τα μεταφέρει στην έξοδο RD. Η μνήμη αυτή είναι μια μνήμη **RAM (Random Access Memory)** με μέγεθος **1024** λέξεων των **32 bit** και θα υλοποιηθεί αρχικά με **SystemVerilog Behavioral**.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (SynchRAM.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας. Η μνήμη να αρχικοποιείται με αρχείο **.txt**.
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

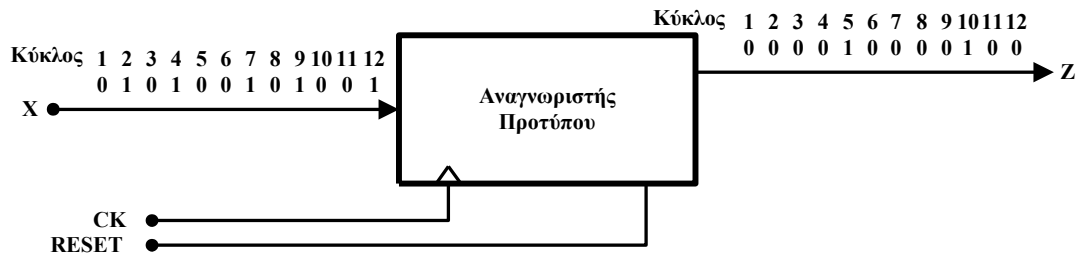


Εικόνα 11. Το δομικό διάγραμμα της Μνήμης Δεδομένων.

Το μνήμη **SynchRAM** να υλοποιηθεί με **Quartus IP Catalog (Altera Megafunctions)** όπως έχουμε διδαχθεί στο μάθημα και να επαναληφθούν όλα τα βήματα προσομοίωσης (**IP SynchRAM.sv**). Η μνήμη να αρχικοποιείται με αρχείο **.mif (MIF – Memory Initialization File)**. Τι παρατηρείται στην υλοποίηση της μνήμης RAM σε σχέση με την πρώτη υλοποίηση;

Μέρος VII

Να σχεδιαστεί το σύγχρονο ακολουθιακό κύκλωμα, που φαίνεται σε μορφή διαγράμματος βαθμίδας στην παρακάτω εικόνα, και θα αναγνωρίζει το πρότυπο (ακολουθία δυαδικών ψηφίων) των τεσσάρων (4) δυαδικών ψηφίων **1010** και θα παράγει μία έξοδο '1' κάθε φορά που το πρότυπο θα παρουσιάζεται σε μία σειριακή είσοδο. Ο αναγνωριστής να υλοποιηθεί σύμφωνα με την ανάλυση που έχετε διδαχθεί στο μάθημα Ψηφιακή Σχεδίαση.



Εικόνα 12. Το διάγραμμα του Αναγνωριστή Προτύπου.

Εκτελέστε τα παρακάτω βήματα για να προσομοιώσετε το κύκλωμα:

1. Δημιουργήστε ένα νέο έργο Quartus Prime για το κύκλωμά σας. Επιλέξτε **Board MAX 10 DE10 - Lite** με το τσιπ προορισμού **MAX 10 (10M50DAF484C6GES)** και ορίστε ως προσομοιωτή τον **Altera-ModelSim**, με γλώσσα περιγραφής **SystemVerilog HDL**.
2. Δημιουργήστε τη σχεδίαση σε ένα αρχείο **SystemVerilog (Rec.sv)** για τον κώδικα και συμπεριλάβετε την στο έργο σας.
3. Υλοποιήστε το κύκλωμα σας (**Analysis & Synthesis**).
4. Με χρήση **SystemVerilog Testbench** του **Quartus Prime/Altera-ModelSim** να προσομοιώσετε τη σχεδίαση σας.
5. **Παραδοτέα:** κώδικες SV (DUT, Testbench), εικόνα από τον Netlist Viewer, εικόνα παραθύρου προσομοίωσης ModelSIM και το compilation report.

Στη συνέχεια ο αναγνωριστής να υλοποιηθεί με **Behavioral SystemVerilog (Recbeh.sv)** και να επαναληφθούν τα βήματα προσομοίωσης.

ΚΑΛΗ ΕΠΙΤΥΧΙΑ!!!

ΠΑΡΑΡΤΗΜΑ

SystemVerilog Coding Styles

Ο τρόπος που περιγράφουμε μια σχεδίαση **SystemVerilog** έχει να κάνει και με το «στυλ» που υιοθετούμε στην συγγραφή και σηματοδοσία σε διάφορα σημεία του κώδικα.

Για το δικό μας εργαστήριο θα πρέπει να υιοθετήσουμε κάποιες συμβάσεις – προθέματα για να διευκολύνουμε την ζωή μας ως Σχεδιαστής Ψηφιακών Συστημάτων.

Υπάρχουν τρία βασικά οφέλη για την υιοθέτηση του στυλ συγγραφής κώδικα, αυτά είναι:

- ✓ Η υψηλή αναγνωσιμότητα του κώδικα. Ο κώδικας είναι εύκολα κατανοητός.
- ✓ Υψηλή συγκέντρωση στη συγγραφή του κώδικα.
- ✓ Ο κώδικας είναι λιγότερο επιρρεπής σε σφάλματα.

Περισσότερες πληροφορίες στον σύνδεσμο: <https://www.systemverilog.io/styleguide>

Προσοχή: Η SystemVerilog είναι case-sensitive!!!

Δήλωση της SystemVerilog ως γλώσσα στο Quartus Prime

Πηγαίνουμε στο μενού του Quartus Prime:

Assignments > Settings > Compiler Settings > Verilog HDL Input

Και επιλέγουμε: "**SystemVerilog**" under Verilog version.

BIBΛΙΟΓΡΑΦΙΑ - ΑΝΑΦΟΡΕΣ

Οι ασκήσεις βασίζονται σε εκπαιδευτικό υλικό από:

1. το ακαδημαϊκό σύγγραμμα «Ψηφιακή σχεδίαση και αρχιτεκτονική υπολογιστών - Έκδοση ARM» των S.Harris και D.Harris, Εκδόσεις ΚΛΕΙΔΑΡΙΘΜΟΣ, 2020.
2. Intel Corporation - FPGA University Program, 2018.
3. Ψηφιακή Σχεδίαση με τις Γλώσσες VHDL και Verilog, Αρχές και Πρακτικές», Δ. Πογαρίδη, Εκδόσεις ΔΙΣΙΓΜΑ, 2019.
4. Nelson Brent, Designing Digital Systems With SystemVerilog (v2.0). Kindle Edition.

Το πρόγραμμα σε ARM Assembly για τις μνήμες.

```
// david_harris@hmc.edu and sarah_harris@hmc.edu
// Test ARM processor
// ADD, SUB, AND, ORR, LDR, STR, B

MAIN    SUB R0, R15, R15    1110 000 0010 0 1111 0000 0000 0000 1111
        ADD R2, R0, #5      1110 001 0100 0 0000 0010 0000 0000 0101
        ADD R3, R0, #12     1110 001 0100 0 0000 0011 0000 0000 1100
        SUB R7, R3, #9      1110 001 0010 0 0011 0111 0000 0000 1001
        ORR R4, R7, R2      1110 000 1100 0 0111 0100 0000 0000 0010
        AND R5, R3, R4      1110 000 0000 0 0011 0101 0000 0000 0100
        ADD R5, R5, R4      1110 000 0100 0 0101 0101 0000 0000 0100
        SUBS R8, R5, R7     1110 000 0010 1 0101 1000 0000 0000 0111
        BEQ END            0000 1010 0000 0000 0000 0000 0000 0000 1100
        SUBS R8, R3, R4     1110 000 0010 1 0011 1000 0000 0000 0100
        BGE AROUND         1010 1010 0000 0000 0000 0000 0000 0000 0000
        ADD R5, R0, #0      1110 001 0100 0 0000 0101 0000 0000 0000
AROUND  SUBS R8, R7, R2     1110 000 0010 1 0111 1000 0000 0000 0010
        ADDLT R7, R5, #1    1011 001 0100 0 0101 0111 0000 0000 0001
        SUB R7, R7, R2      1110 000 0010 0 0111 0111 0000 0000 0010
        STR R7, [R3, #84]   1110 010 1100 0 0011 0111 0000 0101 0100
        LDR R2, [R0, #96]   1110 010 1100 1 0000 0010 0000 0110 0000
        ADD R15, R15, R0    1110 000 0100 0 1111 1111 0000 0000 0000
        ADD R2, R0, #14     1110 001 0100 0 0000 0010 0000 0000 0001
        B END              1110 1010 0000 0000 0000 0000 0000 0000 0001
        ADD R2, R0, #13     1110 001 0100 0 0000 0010 0000 0000 0001
        ADD R2, R0, #10     1110 001 0100 0 0000 0010 0000 0000 0001
END     STR R2, [R0, #100]  1110 010 1100 0 0000 0010 0000 0101 0100
```

Εικόνα 13. Ο κώδικας assembly για το πρόγραμμα που θα φορτωθεί στη μνήμη εντολών.

Και οι κώδικες εντολών σε επεξεργάσιμη μορφή (εσείς πρέπει να το φέρεται σε μορφή που να το διαβάσει η SystemVerilog, κάθε 32bit είναι μια εντολή / μπορείτε να τα μετατρέψετε σε hex):

```
1110 000 0010 0 1111 0000 0000 0000 1111
1110 001 0100 0 0000 0010 0000 0000 0101
1110 001 0100 0 0000 0011 0000 0000 1100
1110 001 0010 0 0011 0111 0000 0000 1001
1110 000 1100 0 0111 0100 0000 0000 0010
1110 000 0000 0 0011 0101 0000 0000 0100
1110 000 0100 0 0101 0101 0000 0000 0100
1110 000 0010 1 0101 1000 0000 0000 0111
0000 1010 0000 0000 0000 0000 0000 1100
1110 000 0010 1 0011 1000 0000 0000 0100
1010 1010 0000 0000 0000 0000 0000 0000
1110 001 0100 0 0000 0101 0000 0000 0000
1110 000 0010 1 0111 1000 0000 0000 0010
1011 001 0100 0 0101 0111 0000 0000 0001
1110 000 0010 0 0111 0111 0000 0000 0010
1110 010 1100 0 0011 0111 0000 0101 0100
1110 010 1100 1 0000 0010 0000 0110 0000
1110 000 0100 0 1111 1111 0000 0000 0000
1110 001 0100 0 0000 0010 0000 0000 0001
1110 1010 0000 0000 0000 0000 0000 0001
1110 001 0100 0 0000 0010 0000 0000 0001
1110 001 0100 0 0000 0010 0000 0000 0001
1110 010 1100 0 0000 0010 0000 0101 0100
```

Παράδειγμα αρχείου αρχικοποίησης μνήμης .mif.

```
DEPTH = 32;
WIDTH = 32;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;
CONTENT
BEGIN
    00      :      00100000001000000010000000100000; % mvi r0,#5  %
    01      :      00100000001000000010000000100000;
    02      :      00100000001000000010000000100000;
    [03..1F]:      00000000000000000000000000000000;
END;
```

ModelSIM – Basic Commands

Βασικές εντολές του προσομοιωτή ModelSim που μπορούν να χρησιμοποιηθούν σε τερματικό παράθυρο (shell) ή στο παράθυρο εντολών (transcript window)

Εντολές δημιουργίας βιβλιοθήκης σχεδίασης και μεταγλώττισης

(Design library creation and design compilation commands)

- vlib** - Create a new design library
- vmap** - Define mapping between logical library name and directory path
- vdir** - List the contents of a design library
- vdel** - Delete a design unit from a specified library
- vmake** - Create a Makefile for a specified design library
- vlog** - Compile Verilog source code into a specified design library
- vsim** - Invoke the VSIM simulator
- verror i** - Prints info about warning/error messages, i is 4-digit msg number

Εντολές Προσομοίωσης (Simulation commands)

- view** - Create windows (wave, list, source etc.)
- add wave** - Add objects to Wave window
- force** - Force a stimulus to Verilog signals
 - > Force input1 to 0 at the current simulator time.
force input1 0
 - > Force the fourth element of the array bus1 to 1 at the current simulator time.
force bus1(4) 1
 - > Force bus1 to 0110 at 100 nanoseconds after the current simulator time.
force bus1 'b0110 100 ns
 - > Create a Clock signal with duty cycle 50% and period 160 ps.
force -repeat 160ps clk 1 0, 0 80
- change** - Change the value of a Verilog variable, constant or generic
- run** - Run simulation
- restart** - Reload and restart the simulation, option -f overrides need for confirmation
- do** - Execute a macro (do) file

Εντολές Αποσφαλμάτωσης (Debug commands)

- examine** - Examine the value of a signal or a variable
- bp** - Set a breakpoint
- disablebp** - Disable breakpoints
- enablebp** - Enable breakpoints
- checkpoint** - Save a simulator state
- restore** - Restore saved simulator state

Περισσότερα στον οδηγό του ModelSIM:

modelsim_ref.pdf (<φάκελος εγκατάστασης>\19.1\modelsim_ase\docs\pdfdocs)

SystemVerilog Syntax – Read from Files

Η SystemVerilog επιτρέπει την αρχικοποίηση μνήμης από αρχείο text με hex ή binary τιμές:

`$readmemh("hex_memory_file.mem", memory_array, [start_address], [end_address])` ή
`$readmemb("bin_memory_file.mem", memory_array, [start_address], [end_address])`

Βασικές δηλώσεις στις συναρτήσεις (Function Arguments):

- **hex_memory_file.mem** - a text file containing hex values separated by whitespace
- **bin_memory_file.mem** - a text file containing binary values separated by whitespace
- **memory_array** - name of SV memory array of the form: logic [n:0] memory_array [0:m]
- **start_address** - where in the memory array to start loading data (optional)
- **end_address** - where in the memory array to stop loading data (optional)

Ένα απλό παράδειγμα testbench με `$readmemh`:

```
module readmemhtb();  
    logic [7:0] testmemory [0:15];  
    initial begin  
        $display("Loading rom.");  
        $readmemh("romimage.mem", testmemory);  
    end  
endmodule
```

Σύνταξη ενός αρχείου Μνήμης

Σε ένα αρχείο text δεδομένων μνήμης (σε hex ή binary) οι τιμές μπορούν να διαχωρίζονται με κενά (space, tab, and newline). Μπορείς να κάνεις συνδυασμούς των κενών σημείων μέσα στο αρχείο. Τα σχόλια δηλώνονται όπως και στην SystemVerilog με τα `//` ξεκινά ένα σχόλιο. Το πλάτος των τιμών των δεδομένων δεν πρέπει να είναι μεγαλύτερο από το πλάτος των δεδομένων του πίνακα, αν αυτό συμβεί τα δεδομένα θα περικόπτονται (truncated). Με το σύμβολο `@(hex)` δηλώνουμε την διεύθυνση του επόμενου δεδομένου στην μνήμη. Μερικά παραδείγματα:

Four 16-bit data values in hex

```
logic [15:0] ex1memory [0:3];  
$readmemh("ex1.mem", ex1memory);  
ex1.mem: dead // this is a comment  
          beef // use of new line for formatting  
          0a0a  
          1234
```

Sixteen 8-bit data values in hex (mixing spaces and newlines)

```
logic [7:0] ex2memory [0:15];  
$readmemh("ex2.mem", ex2memory);  
ex2.mem:  ab cd ef 01 // this is a comment  
          ef 22 1e    // location $7 data not declared  
          @8 9f ff 13 e6 // @8 declares memory location 8 hex  
          ce b7 28 8f
```

Six 3-bit values in binary

```
logic [2:0] ex3memory [0:5];  
$readmemb("ex3.mem", ex3memory);  
ex3.mem: 001 101 111 111 101 001
```

Six 16-bit values in hex starting at array position 4

```
logic [15:0] ex4memory [0:255];  
$readmemh("ex4.mem", ex4memory, 4);  
ex4.mem: dead beef 0a0a 1234 abab 987e
```